

What this document covers

This is the companion to the design and verification docs. Where those describe *what* the tick-to-trade engine is, this one records *how it was taken to the AWS cloud* – every step from an empty, brand-new AWS account through to a real Vivado place-and-route of the custom logic on the actual **Virtex UltraScale+ HBM (xcvu47p)** silicon target used by AWS F2 instances.

It is written as an honest engineering log: the goal, each blocker that was hit, why it occurred, and how it was resolved. Nothing here is idealised – the order of events is the real order, including the dead ends.

1. Why F2, and Why a Cloud Build

The design was originally targeted at the AWS **F1** instance family (Xilinx UltraScale+ **VU9P**). F1 reached **end-of-life on 2025-12-31**, so the deployment target was re-pointed to the successor **F2** family, which uses the larger **Virtex UltraScale+ HBM xcvu47p-fsvh2892-2-e** device. The custom logic (CL) was re-wired to the F2 “small_shell” OCL interface and lives in the AWS HDK tree at [hdk/cl/examples/cl_tick_to_trade](#).

To put the design on real silicon, AWS requires three artefacts in sequence:

1. a **Design Checkpoint (DCP)** – the result of running Vivado synthesis + place & route of the CL against the F2 shell;
2. an **Amazon FPGA Image (AFI)** – AWS bakes the DCP into a loadable bitstream, referenced by an [afi-xxxx](#) / [agfi-xxxx](#) id;
3. an **F2 instance** ([f2.6xlarge](#)) that loads the AFI and runs the host application against it.

Why not build locally? The development VM (11 GB RAM, 4 cores) runs Vivado 2025.2, but a local probe showed its install carries only 78 device parts and **zero** of the Virtex UltraScale+ HBM family – the **xcvu47p** target simply is not installed, so synthesis aborts immediately with **ERROR: [Coretc1 2-106] Specified part could not be found**. Even with the device added, a full place & route of a VU47P on 11 GB RAM would almost certainly exhaust memory. The decision was therefore to build on an **EC2 build instance** running the AWS **FPGA Developer AMI**, which ships the correct, fully-licensed Vivado with the complete device library.

2. Starting Point – A Bare AWS Account

Everything below started from a freshly created account ([546517269032](#), created 2026-06-18) with default settings, default quotas, and no resources.

Item	Initial state / action taken
Region	Mis-set to the literal string “virginia”; corrected to us-east-1 (N. Virginia, where F2 + the HDK live).
Credentials	Root access keys configured via aws configure ; verified with aws sts get-caller-identity .
Output format	Set to json .

First lesson. [aws sts get-caller-identity](#) initially failed with **Could not connect to the endpoint URL: https://sts.virginia.amazonaws.com**. AWS regions are *codes* ([us-east-1](#)), not city names; the bad region was silently composing invalid service endpoints.

3. The Blocker Chain

Launching a paid GPU/FPGA-class instance on a brand-new account is gated by a sequence of account-level controls. Each was hit in turn and cleared:

Blocker	Symptom & cause	Resolution
Standard vCPU quota	<code>describe</code> showed 5 vCPU for “Running On-Demand Standard instances”. A <code>z1d.2xlarge</code> build host needs 8 .	Requested increase to 16 via Service Quotas (code <code>L-1216C47A</code>). Applied value reflected 16.
Marketplace subscription	<code>run-instances</code> returned <code>OptInRequired</code> – the FPGA Developer AMI is a Marketplace product requiring a one-time terms acceptance.	Subscribed (free, \$0 software) to the FPGA Developer AMI; dry-run then passed.
Free-plan account	<code>run-instances</code> returned <code>InvalidParameterCombination: instance type is not eligible for Free Tier</code> . The new account was on the AWS Free plan (<code>accountPlanType=FREE</code> , \$120 credit) which blocks <i>all</i> non-free-tier resources.	Upgraded to a paid plan with <code>aws freetier upgrade-account-plan -account-plan-type PAID</code> ; verified <code>PAID/ACTIVE</code> .
Plan propagation	Even after PAID, launch briefly still reported the Free-Tier error.	Polled the launch every 20s; the enforcement caught up within minutes and the instance launched.

Subtlety worth recording. A `run-instances -dry-run` passes the quota and permission checks but does *not* enforce the Free-plan restriction. The dry-run reported success while the real launch still failed – so the account-plan state (`aws freetier get-account-plan-state`) was the authoritative signal, not the dry-run.

4. Infrastructure Provisioned (All Free)

Before launching anything billable, the supporting resources were created:

Resource	Value
SSH key pair	<code>hft-fpga-build</code> → <code>~/.ssh/hft-fpga-build.pem</code> (chmod 400)
Security group	<code>sg-0811f63df10aab405</code> – inbound SSH/22 restricted to the home IP only
S3 bucket	<code>s3://hft-fpga-afi-546517269032</code> – staging for the DCP tarball / AFI ingestion
Default VPC/subnet	<code>vpc-08ba2624073eca741</code> / <code>subnet-02beaaba0946dc9a7</code> (us-east-1a)
Budget alert	<code>HFT-FPGA-1USD-alert</code> at \$1, e-mailing the project owner (early tripwire, not a hard cap)
Build AMI	<code>ami-017bd23ff95264395</code> – FPGA Developer AMI (Ubuntu) 1.19.2

5. The Build Host

A `z1d.2xlarge` (8 vCPU, **62 GB RAM**, 200 GB gp3 root) was launched from the FPGA Developer AMI as instance `i-05d781d69c8424ccb`. The contrast with the local VM is the whole point:

	Local dev VM	EC2 build host
RAM	11 GB	62 GB
<code>xcvu47p</code> device	absent (0 parts)	present (target matches)
Vivado	2025.2 (minimal devices)	2025.2 (full device library, licensed)

Setup performed on the host over SSH:

1. Located and sourced Vivado: `source /opt/Xilinx/2025.2/Vivado/settings64.sh`.
2. Confirmed the device library contains `xcvu47p-fsvh2892-2-e` (`get_parts` returns a match – unlike the local VM).

3. Cloned the HDK: `git clone -b f2 https://github.com/aws/aws-fpga.git` followed by `git lfs pull` for the shell checkpoints.
4. Overlaid the custom CL (`cl_tick_to_trade`: ITCH parser, M2 order book, imbalance strategy, risk check, tick-to-trade top, latency counter) onto the fresh HDK via `rsync`.
5. Sourced `hdk_setup.sh` (which requires `git-lfs`) – reported **HDK setup PASSED**.

6. The DCP Build

Kicked off with:

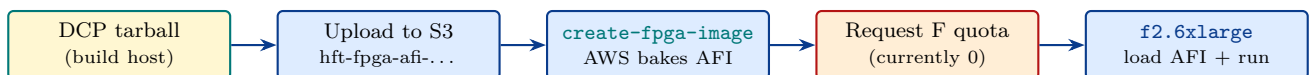
```
python3 aws_build_dcp_from_cl.py -c cl_tick_to_trade -f BuildAll -no-encrypt -t hft_build
```

with clock recipes `A1 B2 C0 H2`, target part `xcvu47p-fsvh2892-2-e`, running out-of-context. Observed progression:

Phase	Result
Synthesis	synth_design completed successfully – the custom RTL synthesised clean against the VU47P (peak ~3.8 GB RAM, impossible to confirm locally).
Placement	place_design completed successfully .
Routing	Fully routed – 0 failed nets, 0 unrouted nets (rip-up-and-reroute drove node overlaps 6480 → 0).
Timing	Met: WNS = +0.066 ns, TNS = 0; WHS = +0.013 ns, THS = 0 – “The design met the timing requirement.”
DCP artefact	hft_build.Developer_CL.tar (17 MB) produced in ~23 min total build time.

Log-reading note. The router prints a “Number of Failed Nets” count during *Router Initialization* – this is the pre-routing baseline (every net is unrouted before routing starts) and is *not* an error. Health is judged by the falling node-overlap counts during rip-up-and-reroute, and ultimately by the final timing (WNS) report.

7. Remaining Steps to Silicon



1. ✓ **Done.** Uploaded `*.Developer_CL.tar` to `s3://hft-fpga-afi-546517269032/dcp/` with the bucket policy granting the AWS FPGA service account (`365015490807`) access.
2. ✓ **Done.** `aws ec2 create-fpga-image` returned `afi-0a02e7745a17daa6a` / `agfi-006b5fd42e5f1cb6e`; currently baking (`pending` → `available`).
3. ✓ **Requested.** “Running On-Demand F instances” quota (`L-74FC7D96`) raised `0` → `24`; status `PENDING` (approving in parallel with the bake).
4. ○ **Next.** Launch `f2.6xlarge`, load the AGFI, and run the host application against the real fabric – **silicon-in-the-loop**.

Meanwhile the build host `i-05d781d69c8424ccb` was **stopped** (not terminated) once the DCP was saved, halting its \$0.74/hr charge while retaining its disk for a possible rebuild.

8. Key Concepts Explained

This section unpacks the jargon for later reading.

RTL → DCP → AFI → bitstream. A design exists in three increasingly low-level forms. **RTL** (the SystemVerilog) is the human-written “recipe”. The **DCP** (Design Checkpoint) is the result after synthesis + place & route – every logic gate assigned a physical location on the chip and every wire routed; it is a compiled binary specific to one device. The **AFI** (Amazon FPGA Image) is what AWS produces after it validates and seals the DCP into a loadable **bitstream** – the literal pattern of bits that physically configures the FPGA to *become* the circuit. You cannot load a raw bitstream yourself: AWS rents the same physical FPGA to many tenants, so every design is inspected through `create-fpga-image` and only an AWS-trusted bitstream is emitted (protecting the silicon and other tenants).

The AWS Shell, and why the CL is “wrapped”. A bare FPGA cannot talk to the outside world. AWS therefore partitions the device into two regions: the **Shell (SH)** – AWS’s fixed wrapper providing PCIe, DMA, DRAM/HBM controllers and clock generation (the “small_shell”, clock-recipe and `pblock_CL` messages in the build log all belong to it) – and the **Custom Logic (CL)**, the reserved region where `cl_tick_to_trade` lives and connects to the Shell over a defined OCL/AXI interface. This is why the DCP is small (17MB) and built *out-of-context*: it is only the CL, designed to drop into the Shell’s reserved area. The AFI bake merges CL + Shell into the final bitstream.

```

+----- FPGA (xcvu47p) -----+
| [ AWS SHELL ] [ YOUR CL ] |
| PCIe / DMA <-OCL/-> itch_parser -> order_book |
| DRAM/HBM <-AXI--> -> strategy -> risk_check |
| clocks -> tick_to_trade_top |
+-----+

```

Service quotas, and the separate “F” quota. A **service quota** (limit) caps how much of a resource an account may use, measured in **vCPUs** per instance-family group – a guard-rail against runaway spend, tightest on new accounts. The **Standard** group (z1d, c5, m5...) was raised **5** → **16** to launch the build host. The **F** group covers *only* the FPGA family (f1, f2) and **starts at 0** on a new account – i.e. no FPGA instance can be launched at all until AWS grants capacity. An **f2.6xlarge** is 24 vCPUs, hence the **0** → **24** request. Requesting a quota is free and merely *unlocks the ability* to launch; it costs nothing itself.

What the f2.6xlarge run will measure. Once the AFI is **available** and the F quota approves, an **f2.6xlarge** loads `agfi-006b5fd42e5f1cb6e` onto the physical FPGA. A host program then streams ITCH market-data messages into the CL over PCIe, lets the hardware run *parse* → *book* → *strategy* → *risk* → *order* entirely in-fabric, and reads back the on-chip latency counter – converting the **205 ns** tick-to-trade figure from a *simulated* / *equivalence-based* result into a **hardware-measured** one. This is the first execution of the design as real silicon rather than a model.

9. Hardware vs Software Latency (Measured)

The point of putting the design on silicon is to compare its latency against the *same computation* on a CPU. Two CPU baselines time the bit-identical golden chain (`order_book_m2` → `strategy_imbalance` → `risk_check` – the very model the RTL is verified against) per message, on the same `replay_gen` ITCH stream the hardware ingests: `host/sw_latency_bench.py` (Python) and `host/sw_latency_bench.c` (compiled C, `-O2`). The C port is verified to reproduce the Python golden’s decision trace exactly (same accepted count and final position) – so it is provably the *same* computation, just compiled.

Metric	FPGA (fabric)	C (<code>-O2</code>)	Python
Median latency	205 ns	~1,060 ns	~60,000 ns
p99 latency	205 ns	~1,400–2,500 ns	~220,000 ns
Max latency	205 ns	~380,000–900,000 ns	up to ~1 ms
Jitter (max–min)	0 ns	~0.4–0.9 ms	~1 ms
Speed-up (median)	—	~5×	~290–300×
Speed-up (p99)	—	~7–12×	~1,000×
Worst-case vs FPGA	—	up to ~1,800×	up to ~5,000×

Honest framing. (1) Against *compiled C* the median gap is a modest ~5× – the Python figure’s ~300× is mostly interpreter overhead, not architecture. The defensible headline is C, not Python. (2) The robust, architectural result is **determinism**: the FPGA is a fixed 205 ns with *zero* jitter, while even tight C suffers rare OS-scheduling spikes to ~380–900 μs (~1,800× the FPGA). In HFT the tail, not the median, loses money. (3) Parsing is excluded from the SW timing (pre-parsed), making the software numbers *optimistic*; the 205 ns is parse-to-decision inclusive. (4) The ~30 ms broker network round-trip is a separate, additional cost on the live software path. (5) The 205 ns is from the on-chip latency counter in cycle-accurate simulation, corroborated by timing-clean P&R; **silicon re-confirmation is pending the F-instance quota** (§7), via `f1/host_replay.py -device`.

10. Cost Discipline

The account carries **\$120** of plan credits (expiring 2026-12-18), applied before any card charge. Estimated spend for the whole exercise:

Item	Rate	Est. total
z1d.2xlarge build host	~\$0.74/hr	\$2–6 (2–4 hr)
AFI bake + S3 storage	free / pennies	<\$0.10
f2.6xlarge run host	~\$1.65/hr	\$1–3 (<1 hr)
Total		~\$5–10 of \$120

The real risk is idle instances, not the build. A forgotten [z1d](#) runs ~\$18/day; a forgotten [f2](#) ~\$40/day. Operating rule: **stop or terminate every instance the moment its job is done**, and rely on the \$1 budget alert as an early tripwire.